Painting with Darts: A Language-Conditioned Tool-Aiming Dataset for Low-Cost Robot Arms*

Vineeth Sendilraj Vardhan Agnihotri Yuvraj Lakhotia

Act-athon 2026, Actor Labs × Physical Intelligence (3rd place)

Abstract

Open robot-learning datasets are dominated by grasping and pick-and-place. We release a dataset for a different class of skill: aiming a handheld, actuated tool at a language-specified target and triggering it. A low-cost SO-101 arm holds the handle of a motorized dart blaster and aims it at a cell of a 5×5 grid given the instruction "shoot ⟨cell⟩". The dataset contains 116 teleoperated demonstrations (29,131 frames, 16 minutes at 30 Hz) with two camera views, joint states, joint-space actions and a binary trigger channel, covering 23 targets, three levels of scene clutter, 17 start poses, and both live and dry shots. Because it differs from common manipulation data in embodiment, task structure and action space, it is suited as out-of-distribution data in training mixtures for generalist policies and as a small benchmark for language-conditioned precision aiming. Fine-tuning $\pi_{0.7}$ on it across 13 configurations, validation error is lowest after about 33 passes (0.287), and every probed checkpoint aims in the commanded row and column order. Combined with a vision-language model (VLM) that plans targets and verifies hits from images, the policy drew pictures with darts in a live demonstration. The data (LeRobot format, CC BY 4.0), code and wiring are public.

1 Introduction

Open robot-learning datasets mostly cover grasping and pick-and-place with a parallel gripper. Many practical skills instead involve holding a tool and pointing it, as in spraying, drilling or aiming a nozzle. These differ from grasping in three ways: success depends on the tool's orientation rather than on contact, the effect lands away from the robot, and the action combines continuous motion with a discrete actuation command. We collected a small dataset of this kind, in which an SO-101 arm aims a motorized dart blaster at a language-specified cell and fires it. We fine-tune the $\pi_{0.7}$ vision-language-action model [1] on these data and pair it with a VLM that plans targets and checks hits, a common hierarchy for long-horizon tasks [2], to draw pictures with darts.

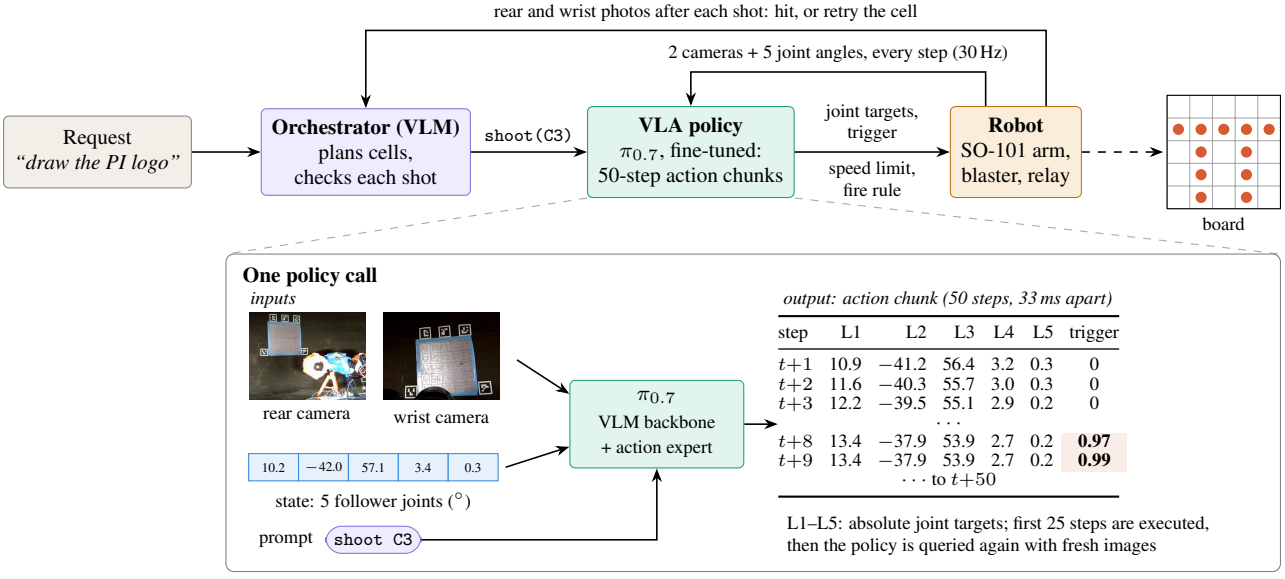


Figure 1. System overview. Top: the VLM decides which cell to shoot next and verifies each result from photographs; the fine-tuned $\pi_{0.7}$ policy turns each `shoot(cell)` command and the live camera stream into joint targets and a trigger at 30 Hz. Bottom: one policy call, with real camera frames from a `shoot B4` demonstration and illustrative joint values.

*Dataset: https://huggingface.co/datasets/vineethsendilraj/pi07-dart-painting-so101. Code, wiring and training logs: https://github.com/VineethSendilraj/pi07-dart-painting-so101.

2 Setup

The follower arm's gripper is replaced by a wire loop around the blaster handle, and an ESP32-driven relay replaces the blaster's trigger switch, so firing is a binary action. Fig. 1 shows the control loop and a single policy call. The policy executes 25 steps of each 50-step chunk before it is queried again; joint speed is capped at 3° per step, and the blaster fires once the arm is still and the predicted trigger has exceeded 0.5 for three consecutive steps. Above the policy, a general-purpose VLM issues one `shoot(cell)` command at a time and re-shoots cells whose dart missed or fell out.

3 Dataset

An operator aimed the follower with a leader arm and fired with a physical button, while a web application proposed the next cell, board clutter and start pose and replayed each take for keep-or-discard review (Fig. 2). Each kept frame holds both images, five follower joint angles, and an action of five leader joint angles plus a trigger that is 1 for the 10 steps after the button press. A5 and E5 were not recorded.

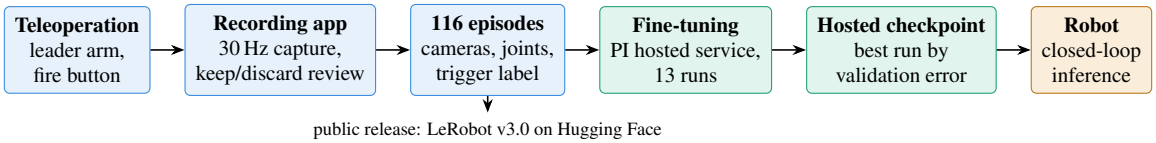


Figure 2. Data and training pipeline. The same episodes are exported to LeRobot format for public release.

4 Training and Results

We ran 13 fine-tuning jobs on Physical Intelligence's hosted service, varying the training schedule, batch size, data mixture and base checkpoint (Table 1). The training schedule mattered most: validation error was lowest after about 33 passes over the data on the standard base checkpoint and about 18 passes on an alternative base, and longer training overfit. A larger batch and oversampling the cells of the PI logo did not help, and training only on the 101 takes recorded with the final crosshair overlay was no better than using all 116. The run with the lowest error on the standard base was used for the live demonstration.

Table 1. Validation error of all 13 fine-tuning runs (best checkpoint per run), by base checkpoint.

Standard base			*Alternative base*		
Change from baseline	Passes	Val. error	Change from baseline	Passes	Val. error
None (baseline)	26.5	0.298	None	26.5	0.294
Longer schedule (demo)	33.5	**0.287**	Shorter schedule	17.6	0.294
Much longer schedule	48.5	0.303	Logo cells oversampled	17.6	0.281[c]
Batch size 512	48.5	0.334	Final-crosshair takes only	∼47	0.294[a]
Logo cells oversampled	33.5	0.333	Still frames dropped	17.6	0.375[b]
Final-crosshair takes only	∼47	0.325[a]	Logo + still frames dropped	17.6	0.444[b]
Still frames dropped	26.5	0.421[b]			

[a] Validated on the 101-take subset. [b] Validated on moving frames only. [c] The validation split likely over-represents the logo cells. Scores marked *a*–*c* are not directly comparable with the others.

To test whether the policy uses the instruction, we gave each checkpoint the same images and joint state while changing only the cell in the prompt. All 24 probed checkpoints produced aims ordered by column and row, so the policy follows the instruction rather than replaying a fixed motion.

5 Limitations

We did not measure a hit rate on the real system; our evidence is validation error, the prompt probe and the live demonstration. Because each demonstration continues for 2 s after the shot with the trigger at 0, the policy rarely produced a clean trigger pulse offline; the stillness rule in Section 2 compensates. Coverage is uneven (two demonstrations each for row 1, A3 and E3), and the camera and board positions were fixed throughout.

References

[1] Physical Intelligence. $\pi_{0.7}$: A steerable generalist robotic foundation model with emergent capabilities. arXiv:2604.15483, 2026.

[2] L. X. Shi, B. Ichter, M. Equi, et al. Hi Robot: Open-ended instruction following with hierarchical vision-language-action models. In *ICML*, 2025.

[3] R. Cadene, S. Alibert, A. Soare, et al. LeRobot: State-of-the-art machine learning for real-world robotics in PyTorch. GitHub, 2024.